

Analyse Big Data avec Hadoop et Spark : Une Approche Moderne et Conteneurisée

Aymen Satouri

October 30, 2025

Contents

1	Introduction : Une Approche Créative	2
1.1	Le Jeu de Données (Dataset)	2
1.2	L'Objectif de l'Analyse	2
2	Environnement : Docker vs. la Machine Virtuelle	2
2.1	Pourquoi je n'ai pas utilisé la VM Cloudera	3
2.2	Ma Solution : Une Image Docker Personnalisée	3
2.3	Mon Workflow : La Stratégie 'Docker Commit'	3
3	Implémentation : Les Quatre Outils d'Analyse	3
3.1	Tâche 1 : MapReduce (Python)	4
3.2	Tâche 2 : Apache Hive	4
3.3	Tâche 3 : Apache Pig	4
3.4	Tâche 4 : Apache Spark	4
4	Résultats Finaux	4
5	Conclusion : Un Outil d'Analyse Portable	6

1 Introduction : Une Approche Créative

Ce projet implémente un pipeline complet d'analyse Big Data en utilisant l'écosystème Hadoop (MapReduce, Hive, Pig) et Apache Spark. Cependant, j'ai voulu adopter une approche plus créative et moderne.

Vous découvrirez pourquoi ce projet est un peu différent. Au lieu de suivre la recommandation d'utiliser l'ancienne VM Cloudera pré-construite, j'ai choisi de bâtir l'environnement de zéro (from scratch) dans un conteneur Docker portable et léger (lightweight). Ce rapport documente le parcours, les défis techniques et l'outil d'analyse final et réutilisable qui a été créé.

1.1 Le Jeu de Données (Dataset)

La source de données est un grand jeu de données (dataset) du monde réel provenant de Kaggle : "1 million de commentaires Reddit de 40 subreddits."

- **Fichier** : `kaggle_RC_2019-05.csv` (renommé en `reddit_comments.csv`)
- **Taille** : 186 MB
- **Format** : Un fichier CSV à 4 colonnes avec les en-têtes : `subreddit`, `body`, `controversiality`, `score`.

Le choix des données Reddit est intéressant car ce sont des données textuelles non structurées et "sales" (dirty) qui reflètent le sentiment réel du public, ce qui en fait un candidat parfait pour l'analyse big data.

1.2 L'Objectif de l'Analyse

L'objectif était de filtrer ce grand dataset pour trouver tous les commentaires mentionnant un ensemble spécifique de mots-clés : `jew`, `jews`, ou `jewish` (insensible à la casse, case-insensitive).

Pour chaque subreddit, nous calculons les métriques suivantes :

1. **Mentions Totales** : Le nombre total de commentaires contenant les mots-clés.
2. **% Controversés** : Le pourcentage de ces commentaires marqués comme controversés (`controversiality == 1`).
3. **% Upvotés** : Le pourcentage avec un score positif (`score > 0`).
4. **% Downvotés** : Le pourcentage avec un score négatif (`score < 0`).

2 Environnement : Docker vs. la Machine Virtuelle

Le brief du projet recommandait d'utiliser la VM Cloudera QuickStart. J'ai fait des recherches sur cette option et j'ai rapidement décidé de ne pas l'utiliser.

2.1 Pourquoi je n'ai pas utilisé la VM Cloudera

La VM Cloudera QuickStart a plus de 8 ans, n'est plus supportée, et est basée sur un produit (CDH 5.13) qui est en fin de vie (end-of-life) depuis de nombreuses années. Même la propre documentation de Cloudera déconseille aux utilisateurs de l'employer. Exécuter un logiciel aussi obsolète et vulnérable semblait être une mauvaise base pour un projet de data moderne.

Je pensais que l'objectif principal de ce projet était d'apprendre les *outils* (MapReduce, Hive, Pig, Spark), et non de se battre avec une machine virtuelle obsolète.

2.2 Ma Solution : Une Image Docker Personnalisée

J'ai décidé d'utiliser Docker, qui est le standard moderne pour le développement portable.

1. J'ai trouvé une image de base : `suhothayan/hadoop-spark-pig-hive:2.9.2`. Cette image contenait tous les services nécessaires dans un environnement léger (lightweight).
2. J'ai démarré le conteneur, en exposant les ports nécessaires (50070 pour HDFS, 8088 pour YARN, etc.), ce qui m'a permis de monitorer HDFS et la progression des jobs depuis le navigateur de ma machine hôte.
3. J'ai chargé mon dataset CSV local dans le conteneur.

2.3 Mon Workflow : La Stratégie 'Docker Commit'

Au début, j'ai monté mon répertoire `/code` et mon fichier CSV en tant que volume Docker. C'est la pratique standard.

Cependant, j'avais peur que si mon conteneur crashait ou était supprimé accidentellement, je perdrais tout mon travail, mes scripts et ma configuration. Les personnes utilisant une VM n'ont pas cette crainte, car l'état de la VM est persistant.

Pour résoudre cela, j'ai adopté une stratégie de "commit".

1. J'ai déplacé les données CSV du volume monté vers le HDFS du conteneur.
2. J'ai gardé tous mes scripts (`mapper.py`, `hive_analysis.sql`, etc.) dans le répertoire `/code` *à l'intérieur* du conteneur.
3. Après avoir terminé une étape avec succès (comme faire fonctionner le job MapReduce), je "committais" l'état actuel du conteneur dans une nouvelle image Docker, par ex., `my-hadoop-dev:v2-mr-done`.

Cela m'a donné des "points de sauvegarde" (save points) et m'a protégé contre la perte de données. Cela a également conduit à l'idée principale de mon projet final : un outil d'analyse portable et entièrement autonome (self-contained).

3 Implémentation : Les Quatre Outils d'Analyse

J'ai implémenté la même analyse en utilisant les quatre outils.

3.1 Tâche 1 : MapReduce (Python)

J'ai choisi d'écrire le job MapReduce natif en Python plutôt qu'en Java. Python est plus moderne, beaucoup plus facile à écrire, et moins sujet aux erreurs de boilerplate. Le job consistait en un `mapper.py` pour filtrer et émettre les métriques, et un `reducer.py` pour les agréger par subreddit.

Ce job a parfaitement fonctionné et a produit notre jeu de résultats "golden standard".

3.2 Tâche 2 : Apache Hive

Hive fut un défi (challenging). Un simple `LOAD DATA` avec `FIELDS TERMINATED BY ','` a échoué, car le corps des commentaires Reddit (colonne 'body') contenait des virgules, des guillemets et des sauts de ligne (newlines).

La solution a été d'utiliser le `OpenCSVSerde`, qui est conçu pour gérer des CSV complexes et "messy". Cela a fonctionné, mais les résultats étaient **légèrement différents** de ceux du job MapReduce natif. Malgré l'ajustement (tweaking) de la requête, je n'ai pas pu obtenir une correspondance exacte.

Je suis convaincu que cela est dû à une différence subtile dans la façon dont le moteur regex `RLIKE` de Hive interprète les délimitations de mots (word boundaries) par rapport au module Python `re`. Les résultats sont très similaires, mais pas identiques.

3.3 Tâche 3 : Apache Pig

Pig a également nécessité des essais et des erreurs (trial and error). À cause du CSV complexe, je ne pouvais pas utiliser le loader natif `PigStorage()`.

La solution a été de réutiliser mon mapper Python ! L'opérateur `STREAM` de Pig permet de "streamer" des données à travers n'importe quel script externe. J'ai streamé les données brutes (raw data) à travers mon script `mapper.py`, dont j'avais déjà prouvé le bon fonctionnement.

Après cela, le script Pig Latin était simple : `GROUP, FOREACH...GENERATE` pour sommer les valeurs, et `STORE` pour les sauvegarder.

Cette approche a parfaitement fonctionné et a produit des résultats identiques à ceux du job MapReduce natif.

3.4 Tâche 4 : Apache Spark

Spark était, de loin, l'outil le plus simple et le plus puissant. En utilisant l'API `DataFrame` de PySpark, j'ai pu lire le fichier CSV complexe correctement en une seule ligne : `spark.read.csv(..., multiLine=True)`

Le seul problème majeur que j'ai rencontré était une `SyntaxError` sur un "f-string" (`f"..."`). J'ai réalisé que c'était parce que `spark-submit` utilisait par défaut l'ancienne version **Python 2.7** du conteneur.

Le correctif (fix) a été de dire explicitement à Spark d'utiliser Python 3 : `PYSPARK_PYTHON=python3 spark-submit spark_analysis.py`

Après cela, le job s'est exécuté sans problème (flawlessly). **Les résultats de Spark étaient également identiques à ceux du job MapReduce natif.**

4 Résultats Finaux

Pour présenter le travail, j'ai organisé les 5 scripts source dans le répertoire `/code`.

J'ai aussi créé un répertoire `/code/FINAL_RESULTS`. Comme le conteneur était lightweight, il n'avait pas l'utilitaire `column`. Je l'ai installé (`apt-get update apt-get install -y bsdmainutils`) et l'ai utilisé pour formater la sortie TSV brute en fichiers texte alignés et "jolis" (pretty-printed) pour une lecture facile.

Ci-dessous se trouvent les commandes pour voir l'en-tête (head) de chaque fichier de résultat.

— 1. Résultats MapReduce (Le "Golden Standard")

```
root@quickstart:/code# cat FINAL_RESULTS/1_mapreduce.txt | head -n 5
```

Subreddit	Total	%_Controversial	%_Upvoted	%_Downvoted
AmItheAsshole	33	9.1	90.9	0.0
Animemes	3	0.0	100.0	0.0
AskReddit	18	0.0	88.9	5.6
ChapoTrapHouse	147	0.0	87.8	10.9

— 2. Résultats Hive (Légèrement Différents)

```
root@quickstart:/code# cat FINAL_RESULTS/2_hive.txt | head -n 5
```

Subreddit	Total	%_Controversial	%_Upvoted	%_Downvoted
news	182	9.3	79.7	15.4
todayilearned	161	6.2	80.1	10.6
ChapoTrapHouse	147	0.0	87.8	10.9
worldnews	107	12.1	78.5	12.1

— 3. Résultats Pig (Identiques à MapReduce)

```
root@quickstart:/code# cat FINAL_RESULTS/3_pig.txt | head -n 5
```

Subreddit	Total	%_Controversial	%_Upvoted	%_Downvoted
news	182	9.3	79.7	15.4
todayilearned	161	6.2	80.1	10.6
ChapoTrapHouse	147	0.0	87.8	10.9
worldnews	107	12.1	78.5	12.1

— 4. Résultats Spark (Identiques à MapReduce)

```
root@quickstart:/code# cat FINAL_RESULTS/4_spark.txt | head -n 5
```

Subreddit	Total	%_Controversial	%_Upvoted	%_Downvoted
news	182	9.3	79.7	15.4
todayilearned	161	6.2	80.1	10.6
ChapoTrapHouse	147	0.0	87.8	10.9
worldnews	107	12.1	78.5	12.1

(Note : La sortie MapReduce n'est pas triée par défaut, alors que les autres le sont. Les données réelles sont identiques, comme confirmé en comparant les fichiers.)

5 Conclusion : Un Outil d'Analyse Portable

Ce projet est un succès. L'analyse a été complétée en utilisant quatre méthodes différentes, et les défis de chacune ont été documentés.

L'image Docker finale et "propre" (clean) (`my-hadoop-dev:v2-clean`) est le véritable livrable (deliverable). Elle contient :

- Les données HDFS originales (`/user/root/project/reddit_comments.csv`)
- Les 5 scripts de code source dans `/code`
- Les 4 fichiers de résultats formatés dans `/code/FINAL_RESULTS`

Cette image est maintenant une "appliance d'analyse" portable et autonome (self-contained). Je pourrais uploader cette unique image sur Docker Hub, et n'importe qui pourrait la télécharger et exécuter mon analyse en quelques secondes.

Plus important encore, c'est maintenant un outil réutilisable. N'importe qui peut prendre cette image, changer une ligne dans le fichier `mapper.py` (pour remplacer "jew" par "trump", "biden", ou "crypto" ou "Satouri" (: ou "Easter Egg", par exemple), ré-exécuter les jobs, et effectuer une nouvelle analyse sur n'importe quel dataset Reddit qui suit le même format. C'est la puissance d'un workflow moderne et conteneurisé.